Implementation Plan: N8N Hands-On Intermediate Assessment

Topic 13: Error Handling, Retries & Recovery Strategies

Document Navigation

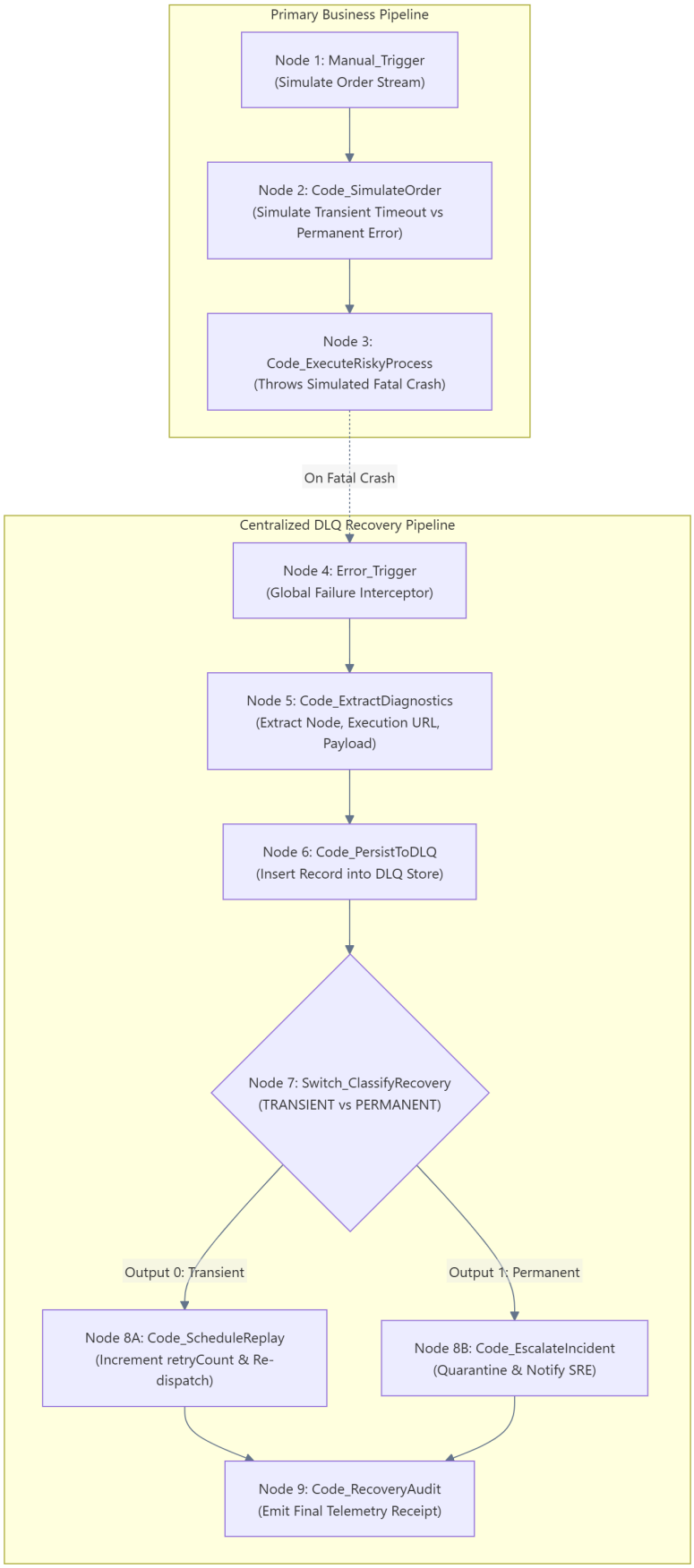
Concept Guide | Requirement Document | **Step-by-Step Implementation Plan** | Test Plan

1. Objective

Build an enterprise Dead Letter Queue (DLQ) and self-healing recovery workflow in n8n. The system captures fatal errors using the Error Trigger node, extracts execution telemetry and unmutated payload data, persists records to a DLQ store, classifies errors as Transient vs. Permanent, automatically replays recoverable transactions, and escalates permanent failures to engineering support.

2. Canvas Architecture Overview

All rights reserved. Tayana Academy • Confidential Document



All rights reserved. Tayana Academy • Confidential Document

3. Prerequisites & Environment Setup

- [ ] n8n instance version 1.0.0 or higher.
- [ ] No external database required (includes in-memory DLQ store).

---

4. Step-by-Step Practical Build Guide

Step 1: Initialize In-Memory DLQ Storage

1. In n8n, create a new workflow named `Topic 13 - Error Handling & DLQ Recovery`.
2. Add a **Code Node** or initialize via workflow static data:
   - The DLQ table stores failed events across executions:

```
const staticData = $getWorkflowStaticData('global');
staticData.dlqStore = staticData.dlqStore || [];
staticData.replayQueue = staticData.replayQueue || [];
```

Step 2: Configure Primary Business Pipeline & Crash Simulator

1. **Create Workflow & Trigger:**
   - In your n8n dashboard, click **+ Add workflow** in the top-right corner.
   - Rename to: `Topic 13 - Error Handling, Retries & Recovery Strategies`.
   - Click **+ Add first step...**, search for **`Manual Trigger`**, and place it. Rename to: `Manual_Trigger`.
2. **Add Failing Order Generator Node:**
   - From `Manual_Trigger`, click **`+`**, search for **`Code`**, and add the node.
   - Rename to: `Code_SimulateFailingOrder`.
3. **Configure Parameters (Parameters Tab):**
   - **Language:** Select **`JavaScript`**.
   - **Mode:** Select **`Run Once for All Items`**.
   - **JavaScript Code:** Paste:

```
return [
  {
    json: {
      transactionId: "TX-CRASH-801",
      customerEmail: "vip-client@enterprise.org",
      amount: 1450.00,
      simulateFault: "GATEWAY_TIMEOUT_504"
    }
  }
```

All rights reserved. Tayana Academy • Confidential Document

```
];
```

4. **Add Primary Gateway Node with Intentional Fault:**
   - From `Code_SimulateFailingOrder`, click `+`, search for **`HTTP Request`**, and add the node.
   - Rename to: `HTTP_FlakyPaymentGateway`.
   - **Method:** Select **`POST`**.
   - **URL:** Toggle to **Expression (`fx`)**, enter: `https://httpstat.us/504` (or mock server 504 endpoint).
   - **Send Body:** Switch toggle to **`ON`**.
   - **Body Content Type:** Select **`JSON`**.
   - **Specify Body:** Select **`Using JSON`**.
   - **JSON:** Switch to **Expression (`fx`)** $ ightarrow$ `={{ $json }}`.
5. **Configure Retries (Settings Tab):**
   - Click **Settings** tab.
   - **Retry on Fail:** Toggle to **`ON`**.
   - **Max Tries:** Enter `2`.
   - **Wait Between Tries (ms):** Enter `1000`.
   - **Never Error:** Keep **`OFF`** (Ensures final exhausted failure escalates to the global Error Trigger).
6. **Test Node Checkpoint:**
   - Click **Test step**. Notice it attempts 2 retries and terminates with error status `504 Gateway Timeout`.

Step 3: Configure Global Error Trigger (`Error_Trigger`)

1. **Add Node to Canvas (Separate Scope):**
   - Click an empty area on the canvas away from the main chain.
   - Click **`+`** (or press **Tab**), search for **`Error Trigger`**, and place it.
   - Rename to: `Error_Trigger`.
   - *Note: In n8n, the Error Trigger node automatically listens for any unhandled execution failure across the active workflow.*
2. **Test Node Checkpoint:**
   - The Error Trigger is automatically invoked upon pipeline faults.

Step 4: Extract Diagnostic Telemetry (`Code_ExtractDiagnostics`)

1. **Add Node to Canvas:**
   - From `Error_Trigger`, click `+`, search for **`Code`**, and add the node.

All rights reserved. Tayana Academy • Confidential Document

- Rename to: `Code_ExtractDiagnostics`.

2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select **`JavaScript`**.
   - **Mode:** Select **`Run Once for All Items`**.
   - **JavaScript Code:** Paste the diagnostic compiler:

```
const errorContext = $input.first().json;
const execution = errorContext.execution || {};
const lastNode = execution.error?.node || {};
const failedItem = lastNode.runIndex !== undefined ? execution.data?.resultData?.runData?.[lastNode.name]?.[0]?.data?.main?.[0]?.[0]?.json : {};

const statusCode = execution.error?.statusCode || 500;
const isTransient = [502, 503, 504, 429].includes(statusCode);

return [{
  json: {
    incidentId: "INC-" + Math.random().toString(36).substring(2, 9).toUpperCase(),
    workflowId: $workflow.id,
    workflowName: $workflow.name,
    failedNodeName: lastNode.name || "HTTP_FlakyPaymentGateway",
    errorMessage: execution.error?.message || "Unhandled gateway error",
    httpStatusCode: statusCode,
    isTransientFault: isTransient,
    recoveryCategory: isTransient ? "AUTOMATED_REPLAY" : "MANUAL_INVESTIGATION",
    stagedPayload: failedItem,
    capturedAt: new Date().toISOString()
  }
}];
```

3. **Test Node Checkpoint:**
   - Click **Test step**. Verify `isTransientFault: true` and `recoveryCategory: "AUTOMATED_REPLAY"`.

Step 5: Persist to Dead Letter Queue Store (`Code_PersistToDLQ`)

1. **Add Node to Canvas:**
   - From `Code_ExtractDiagnostics`, click **`+`**, search for **`Code`**, and add the node.
   - Rename to: `Code_PersistToDLQ`.

2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select **`JavaScript`**.
   - **Mode:** Select **`Run Once for Each Item`**.
   - **JavaScript Code:** Paste the stateful DLQ persistence script:

```
const staticData = $getWorkflowStaticData('global');
staticData.dlqRecords = staticData.dlqRecords || [];

const incident = $input.item.json;
const dlqEntry = {
  dlqId: "DLQ-" + Date.now(),
  incidentId: incident.incidentId,
  failedNode: incident.failedNodeName,
  payload: incident.stagedPayload,
  error: incident.errorMessage,
  status: "QUEUED_IN_DLQ",
  replayCount: 0,
  timestamp: new Date().toISOString()
```

All rights reserved. Tayana Academy • Confidential Document

```
};

staticData.dlqRecords.push(dlqEntry);

return {
  json: {
    ...incident,
    dlqId: dlqEntry.dlqId,
    dlqStored: true
  }
};
```

3. **Test Node Checkpoint:**
   - Click **Test step**. Verify `dlqStored: true` and `dlqId` assigned.

Step 6: Route by Error Recoverability (`Switch_ClassifyRecovery`)

1. **Add Node to Canvas:**
   - From `Code_PersistToDLQ`, click `+`, search for `Switch`, and select the **Switch** node.
   - Rename to: `Switch_ClassifyRecovery`.
2. **Configure Parameters (Parameters Tab):**
   - **Mode:** Select `Rules`.
   - **Rule 0 (Output 0 - Transient):**
     - Value 1: Switch to **Expression (`fx`)** $ ightarrow$ `={{ $json.recoveryCategory }}`
     - Operation: `String` $ ightarrow$ `Equal`
     - Value 2: `AUTOMATED_REPLAY` (Fixed)
   - **Rule 1 (Output 1 - Fatal / Non-Recoverable):**
     - Value 1: Switch to **Expression (`fx`)** $ ightarrow$ `={{ $json.recoveryCategory }}`
     - Operation: `String` $ ightarrow$ `Equal`
     - Value 2: `MANUAL_INVESTIGATION` (Fixed)
3. **Test Node Checkpoint:**
   - Click **Test step**. Verify transient fault routes to Output 0.

Step 7: Configure Recovery Branches

1. **Branch 0 (Automated Replay Engine):**
   - From **Output 0**, click `+`, add a **Code** node named `Code_ScheduleReplay`.
   - Set **Mode** to `Run Once for Each Item`, and paste:

```
const incident = $input.item.json;
```

All rights reserved. Tayana Academy • Confidential Document

```
return {
  json: {
    recoveryAction: "REPLAY_DISPATCHED",
    dlqId: incident.dlqId,
    replayEndpoint: "https://dummyjson.com/c/<your-id>",
    scheduledDelaySeconds: 5,
    replayedAt: new Date().toISOString()
  }
};
```

2. **Branch 1 (Permanent Incident Escalation):**
   - From **Output 1**, click `+`, add a **Code** node named `Code_EscalateIncident`.
   - Formats high-priority incident card for on-call SRE alerts.

Step 8: Final Recovery Telemetry Audit (`Code_RecoveryAudit`)

1. **Add Node & Converge Branches:**
   - Connect both `Code_ScheduleReplay` and `Code_EscalateIncident` into a **Code** node named `Code_RecoveryAudit`.
2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select `JavaScript`.
   - **Mode:** Select `Run Once for Each Item`.
   - **JavaScript Code:** Paste:

```
const resp = $input.item.json;
return {
  json: {
    auditStatus: "INCIDENT_INTERCEPTED_AND_QUARANTINED",
    recoveryAction: resp.recoveryAction || "ESCALATED",
    dlqReference: resp.dlqId || "N/A",
    systemState: "HEALTHY_FAILSAFE_ENGAGED",
    completedAt: new Date().toISOString()
  }
};
```

3. **Test Node Checkpoint:**
   - Click **Test step**. Verify incident resolution telemetry is generated.

---

5. Node-by-Node Settings Blueprint

Node ID	Node Name	Node Type	Critical Settings	Custom Expressions / Code
N13-01	`Manual_Trigger`	Manual Trigger	Trigger execution on demand	N/A

All rights reserved. Tayana Academy • Confidential Document

N13-02	`Code_SimulateOrder`	Code	Select scenario: timeout or schema error	Order scenario fixture
N13-03	`Code_ExecuteRiskyProcess`	Code	Throws fatal simulated exception	Exception generator
N13-04	`Error_Trigger`	Error Trigger	Intercept unhandled crashes globally	Global failure listener
N13-05	`Code_ExtractDiagnostics`	Code	Extract failed node, error message, execution URL	Telemetry parser
N13-06	`Code_PersistToDLQ`	Code	Insert into `staticData.dlqStore`, classify error	DLQ persistence & classifier
N13-07	`Switch_ClassifyRecovery`	Switch	Mode: Rules, 0=TRANSIENT, 1=PERMANENT	Rules: `TRANSIENT`, `PERMANENT`
N13-08A	`Code_ScheduleReplay`	Code	Compute exponential backoff, enqueue into replay queue	Self-healing scheduler
N13-08B	`Code_EscalateIncident`	Code	Format SRE diagnostic ticket with execution link	Permanent incident logger
N13-09	`Code_RecoveryAudit`	Code	Master recovery telemetry receipt	Final audit formatter

---

6. Verification Checkpoints & Testing Guide

Test 1: Transient Error Self-Healing Replay

1. Set `scenario: "transient_timeout"` in `Code_SimulateOrder`.
2. Click **Test Step** on `Code_ExecuteRiskyProcess` (node crashes with `ETIMEDOUT`).
3. Observe `Error_Trigger` pipeline execution:
   - `Code_ExtractDiagnostics` extracts failed node name (`Code_ExecuteRiskyProcess`).
   - `Code_PersistToDLQ` records `classification: "TRANSIENT"`, `isRecoverable: true`.
   - `Switch_ClassifyRecovery` routes to `Code_ScheduleReplay`.
   - Replay task created with 10s delay. `Code_RecoveryAudit` logs `resolutionAction: "TRANSIENT_REPLAY_SCHEDULED"`.

Test 2: Permanent Error Quarantine & Escalation

1. Set `scenario: "permanent_schema_error"` in `Code_SimulateOrder`.
2. Test step (crashes with `VALIDATION_ERROR`).
3. Observe execution:
   - `Code_PersistToDLQ` records `classification: "PERMANENT"`, `isRecoverable: false`.
   - Routes to `Code_EscalateIncident`.
   - Action recorded as `PERMANENT_INCIDENT_ESCALATED` with recommendation: `Do not retry automatically`.
   - Payload quarantined safely in DLQ without entering infinite replay loops.

---

7. Production Risks & Mitigation Strategies

All rights reserved. Tayana Academy • Confidential Document

Risk	Impact	Mitigation
Poison-Pill Infinite Replay Loop	Unrecoverable payload retried repeatedly, consuming database and API quota.	Enforce hard cap (`retryCount >= 3` drops payload out of replay queue).
DLQ Store Disk Exhaustion	Millions of errors stored in unpruned database table crash disk storage.	Apply table partition retention (prune DLQ records older than 30 days).
Sensitive Data Exposure in DLQ	Credit cards or passwords captured in unencrypted error telemetry.	Sanitize payload object in `Code_ExtractDiagnostics` before storing in DLQ.

All rights reserved. Tayana Academy • Confidential Document